



Assignment on Spam Email Detection Using Naive Bayes and Random Forest Classifier

Course: CSE 411

Course Title: Data Science

Session: 2021-2022

Submitted By

Md Ashraful Islam

Roll: 22102001

Reg: 10807

Supervised By

Prof. Dr. Tushar Kanti Saha

Department of CSE

Jatiya Kabi Kazi Nazrul Islam

University

Jatiya Kabi Kazi Nazrul Islam University

Department of Computer Science and Engineering

Trishal, Mymensingh

April 28, 2026

Contents

1	Introduction	1
1.1	Overview	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Assignment Requirement	2
2	Dataset and Exploratory Data Analysis	3
2.1	Dataset Description	3
2.2	Dataset Summary	3
2.3	Class Distribution	4
2.4	EDA Analysis	4
2.5	EDA Figures	5
3	Methodology	7
3.1	System Workflow	7
3.2	Train-Test Split	8
3.3	TF-IDF Vectorization	8
3.4	Naive Bayes Classifier	8
3.5	Random Forest Classifier	9
3.6	Methodology Summary	10
4	Implementation	11
4.1	Development Environment	11
4.2	Project Directory Structure	11
4.3	Naive Bayes Training Code	12
4.4	Naive Bayes Prediction Code	15
4.5	Random Forest Training Code	16
4.6	Random Forest Prediction Code	20
5	Output and Results	22
5.1	Naive Bayes Training Output	22

5.2	Naive Bayes Classification Report	22
5.3	Naive Bayes Confusion Matrix	23
5.4	Random Forest Training Output	23
5.5	Random Forest Classification Report	24
5.6	Random Forest Confusion Matrix	24
5.7	Sample Prediction Output	25
6	Comparative Analysis	26
6.1	Performance Comparison	26
6.2	Analysis and Discussion	26
6.3	When to Use Which Model	27
7	Conclusion	28
7.1	Concluding Remarks	28
7.2	Limitations	28
7.3	Future Scopes	29

Chapter 1

Introduction

1.1 Overview

Email is one of the most widely used communication methods for academic, professional, and personal purposes. However, a large number of unwanted and harmful emails are sent every day. These unwanted emails are generally known as spam emails. Spam emails may contain advertisements, fake offers, fraud links, lottery messages, phishing attempts, or harmful attachments.

The objective of this assignment is to develop machine learning-based spam email detection systems using two different classifiers: Naive Bayes and Random Forest. Both models are trained on the same dataset and their performances are compared to identify which classifier is more effective for spam email detection.

1.2 Problem Statement

Manual identification of spam emails is time-consuming and inefficient. A user may receive many emails every day, and it is difficult to manually check each email. Therefore, an automated system is required to classify emails as spam or not spam. The main problem of this assignment is to design and implement machine learning models that can learn from labeled email data and predict the class of new email messages. Two models are developed and compared: Multinomial Naive Bayes and Random Forest.

1.3 Objectives

The major objectives of this assignment are:

- To load and analyze the provided spam email dataset.
- To apply Exploratory Data Analysis (EDA) on the dataset.
- To preprocess email text data for machine learning.

- To convert text into numerical features using TF-IDF vectorization.
- To train a Naive Bayes classifier using an 80:20 train-test split.
- To train a Random Forest classifier using the same dataset and split.
- To evaluate both models using accuracy, classification report, and confusion matrix.
- To compare the performance of both classifiers.
- To test the trained models with new email messages.

1.4 Assignment Requirement

The assignment instruction was to check whether an email is spam or not using machine learning classifiers and the provided dataset. This report includes dataset analysis, methodology, implementation code, output, model explanation, result analysis, and a comparative discussion of Naive Bayes and Random Forest classifiers.

Chapter 2

Dataset and Exploratory Data Analysis

2.1 Dataset Description

The dataset used in this assignment is named `emails.csv`. It contains email text and the corresponding class label. The dataset has two columns: `text` and `spam`. The `text` column contains the email message, while the `spam` column represents the target label. The same dataset is used for both the Naive Bayes and Random Forest models.

Table 2.1: Dataset Column Description

Column Name	Data Type	Description
text	String	Contains the complete email message.
spam	Integer	Target class label where 1 means spam and 0 means not spam.

2.2 Dataset Summary

The dataset contains a total of 5,728 email records. Among them, 4,360 emails are not spam and 1,368 emails are spam. This shows that the dataset is imbalanced, because the number of non-spam emails is higher than the number of spam emails.

Table 2.2: Dataset Summary

Property	Value
Dataset File	emails.csv
Total Records	5728
Total Columns	2
Text Column	text
Label Column	spam
Missing Values	0

2.3 Class Distribution

Table 2.3: Class Distribution of Emails

Class Label	Meaning	Number of Emails
0	Not Spam	4360
1	Spam	1368

The class distribution shows that most of the emails in the dataset are not spam. Although the dataset is not perfectly balanced, it contains enough spam and non-spam examples for training a supervised classification model.

2.4 EDA Analysis

Exploratory Data Analysis was performed to understand the structure and quality of the dataset. The EDA process included checking the dataset shape, column names, missing values, label distribution, email length, and word count distribution.

Table 2.4: Email Length and Word Count Statistics

Statistic	Email Length	Word Count
Count	5728	5728
Mean	1556.77	326.85
Standard Deviation	2042.65	418.78
Minimum	13	2
25%	508.75	101
50%	979	210
75%	1894.25	401
Maximum	43952	8477

The EDA result shows that email messages vary significantly in length. Some emails are very short, while some are extremely long. This variation is important because spam emails often contain promotional text, repeated phrases, or long advertising content. On the other hand, non-spam emails may contain academic or official messages with different writing styles.

2.5 EDA Figures

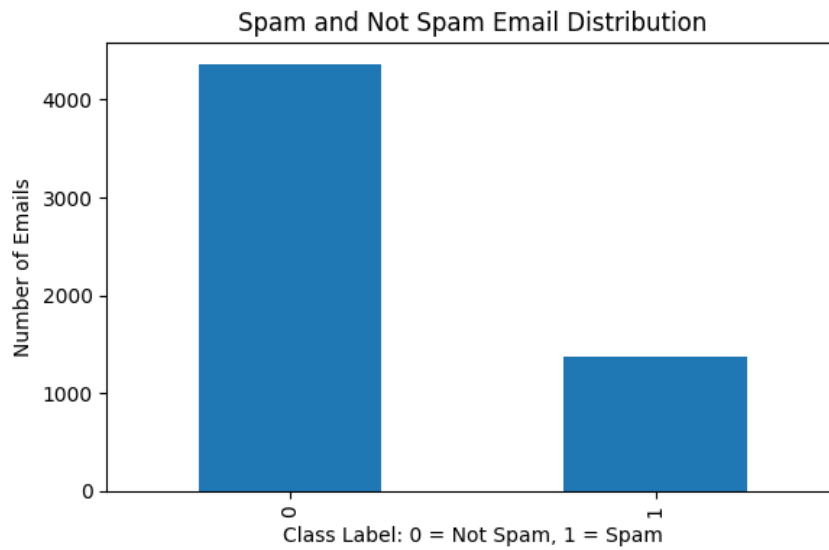


Figure 2.1: Spam and Not Spam Email Distribution

Figure 2.1 shows the distribution of spam and non-spam emails. The chart indicates that non-spam emails are more frequent than spam emails in the dataset.

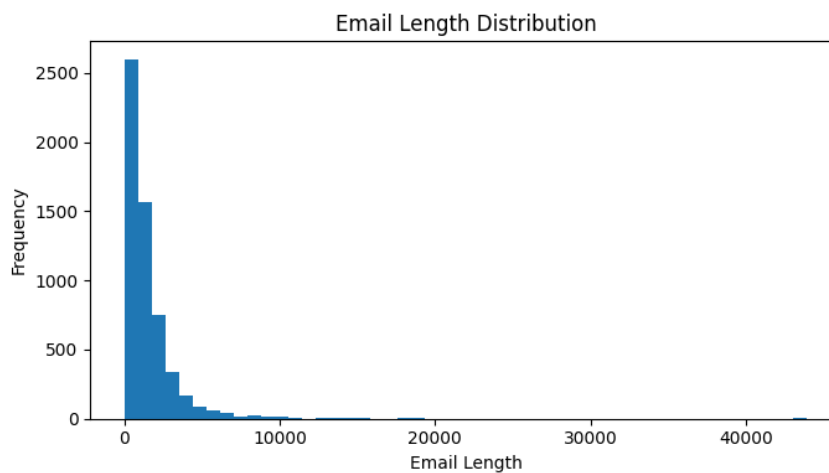


Figure 2.2: Email Length Distribution

Figure 2.2 shows the distribution of email length. This helps to understand how long or short the email messages are.

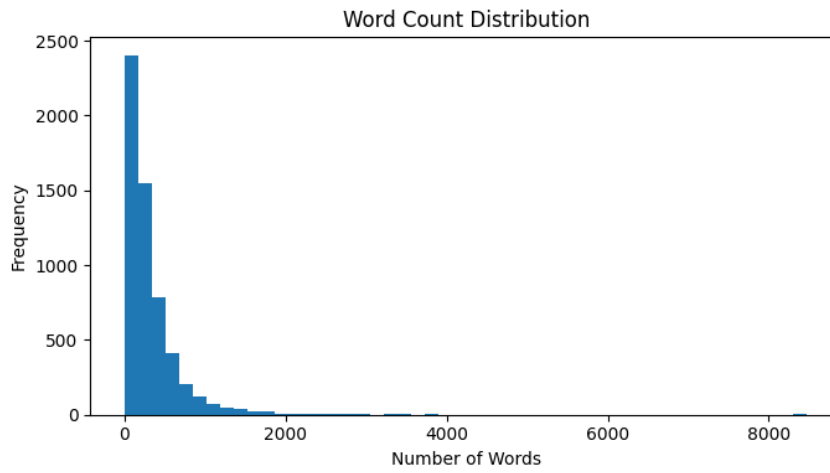


Figure 2.3: Word Count Distribution

Figure 2.3 shows the word count distribution. Word count is useful for analyzing the text structure of emails.

Chapter 3

Methodology

3.1 System Workflow

Both the Naive Bayes and Random Forest spam detection systems follow the same structured machine learning workflow. First, the dataset is loaded. Then EDA is applied to understand the data. After that, the text data is converted into numerical features using TF-IDF. Finally, each classifier is trained and evaluated independently.

Table 3.1: Machine Learning Workflow (Common for Both Models)

Step	Process	Description
1	Dataset Loading	The <code>emails.csv</code> file is loaded using <code>pandas</code> .
2	EDA	Dataset shape, missing values, class distribution, email length, and word count are analyzed.
3	Data Cleaning	Missing values are removed and labels are converted into integer format.
4	Train-Test Split	The dataset is divided into 80% training data and 20% testing data.
5	Feature Extraction	TF-IDF is used to convert email text into numerical feature vectors.
6	Model Training	Classifier (NB or RF) is trained using training data.
7	Model Evaluation	Accuracy, classification report, and confusion matrix are generated.
8	Prediction	New email text is classified as spam or not spam.

3.2 Train-Test Split

The dataset was divided into 80% training data and 20% testing data for both models. The same split and random state were used to ensure a fair comparison.

Table 3.2: Train-Test Split

Data Part	Number of Records
Training Data	4582
Testing Data	1146
Total Data	5728

3.3 TF-IDF Vectorization

Machine learning models cannot directly understand raw text. Therefore, the email text must be converted into numerical form. In this project, TF-IDF vectorization was used for both models. TF-IDF stands for Term Frequency-Inverse Document Frequency. It gives importance to words that are frequent in a particular email but not common in all emails.

The TF-IDF value is calculated as:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

where,

$$IDF(t) = \log \left(\frac{N}{DF(t)} \right)$$

Here, $TF(t, d)$ is the frequency of term t in document d , N is the total number of documents, and $DF(t)$ is the number of documents containing term t .

3.4 Naive Bayes Classifier

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' theorem. It is widely used for text classification problems such as spam detection because it performs well with word-frequency based features.

Bayes' theorem is written as:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

where,

- $P(C|X)$ is the posterior probability of class C given input X .
- $P(X|C)$ is the likelihood of input X given class C .
- $P(C)$ is the prior probability of class C .
- $P(X)$ is the evidence or probability of input X .

The final class is selected using the following decision rule:

$$\hat{C} = \arg \max_C P(C) \prod_{i=1}^n P(x_i|C)$$

To avoid zero probability for unseen words, Laplace smoothing is applied:

$$P(x_i|C) = \frac{\text{count}(x_i, C) + \alpha}{\sum_{j=1}^n \text{count}(x_j, C) + \alpha n}$$

where $\alpha = 0.1$ was used in this project.

3.5 Random Forest Classifier

Random Forest is an ensemble learning algorithm that builds multiple decision trees during training and combines their outputs to produce the final prediction. It is robust against overfitting and performs well on high-dimensional text features.

The Random Forest classifier works as follows. Given a training dataset, it constructs T decision trees, where each tree is trained on a random bootstrap sample of the data. At each node of a tree, only a random subset of features is considered for splitting. The final prediction is made by majority voting across all trees:

$$\hat{C} = \arg \max_C \sum_{t=1}^T \mathbb{I}[h_t(x) = C]$$

where $h_t(x)$ is the prediction of the t -th tree for input x , and $\mathbb{I}[\cdot]$ is the indicator function. In this project, $T = 100$ trees were used with `random_state = 42`.

3.6 Methodology Summary

Table 3.3: Methodology Summary for Both Models

Component	Naive Bayes	Random Forest
Programming Language	Python	Python
Dataset	emails.csv	emails.csv
EDA Library	pandas, matplotlib	pandas, matplotlib
Feature Extraction	TF-IDF Vectorizer	TF-IDF Vectorizer
Classifier	MultinomialNB	RandomForestClassifier
Key Parameter	alpha = 0.1	n_estimators = 100
Training Data	80%	80%
Testing Data	20%	20%
Evaluation Metrics	Accuracy, Precision, Recall, F1, Confusion Matrix	Accuracy, Precision, Recall, F1, Confusion Matrix, ROC-AUC
Model Saving Tool	joblib	joblib

Chapter 4

Implementation

4.1 Development Environment

Both projects were implemented using Python in Visual Studio Code. A virtual environment was used to manage the required Python packages.

Table 4.1: Development Environment

Tool/Technology	Purpose
Python 3	Programming language
VS Code	Code editor
pandas	Dataset loading and analysis
matplotlib	EDA visualization
scikit-learn	Machine learning model development
joblib	Saving and loading trained model
numpy	Numerical operations for RF figures

4.2 Project Directory Structure

```
Spam Email Detector/  
|  
|-- emails.csv  
|-- README.md  
|-- requirements.txt  
|-- .gitignore  
|  
|-- Naive Bayes/  
|   |-- train_model.py  
|   |-- predict.py  
|   |-- spam_email_detector.pkl  
|   |-- figures/  
|       |-- label_distribution.png
```

```

|         |-- email_length_distribution.png
|         |-- word_count_distribution.png
|         |-- confusion_matrix.png
|
|-- Random Forest/
|   |-- train_rf_model.py
|   |-- predict_rf.py
|   |-- spam_rf_model.pkl
|   |-- figures/
|       |-- confusion_matrix.png
|       |-- feature_importance.png
|       |-- roc_curve.png
|       |-- learning_curve.png
|
|-- venv/

```

4.3 Naive Bayes Training Code

```

import os
import pandas as pd
import joblib
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score,
    classification_report, confusion_matrix

BASE_DIR      = os.path.dirname(os.path.abspath(__file__))
DATASET_PATH  = os.path.join(BASE_DIR, "..", "emails.csv")
FIGURE_DIR   = os.path.join(BASE_DIR, "figures")
TEXT_COL      = "text"
LABEL_COL     = "spam"

def create_directory():
    if not os.path.exists(FIGURE_DIR):
        os.makedirs(FIGURE_DIR)

```

```

def perform_eda(df):
    print("\n===== Exploratory Data Analysis =====")
    print("\nDataset Shape:"); print(df.shape)
    print("\nDataset Columns:"); print(df.columns.tolist())
    print("\nFirst 5 Rows:"); print(df.head())
    print("\nMissing Values:"); print(df.isnull().sum())
    print("\nLabel Distribution:"); print(df[LABEL_COL].
        value_counts())

    df["email_length"] = df[TEXT_COL].astype(str).apply(len)
    df["word_count"]    = df[TEXT_COL].astype(str).apply(
        lambda x: len(x.split()))

    print("\nEmail Length Statistics:")
    print(df["email_length"].describe())
    print("\nWord Count Statistics:")
    print(df["word_count"].describe())

    plt.figure(figsize=(6, 4))
    df[LABEL_COL].value_counts().plot(kind="bar")
    plt.title("Spam and Not Spam Email Distribution")
    plt.xlabel("Class Label: 0 = Not Spam, 1 = Spam")
    plt.ylabel("Number of Emails")
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURE_DIR, "label_distribution.
        png"))
    plt.close()

    plt.figure(figsize=(7, 4))
    plt.hist(df["email_length"], bins=50)
    plt.title("Email Length Distribution")
    plt.xlabel("Email Length"); plt.ylabel("Frequency")
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURE_DIR, "
        email_length_distribution.png"))
    plt.close()

    plt.figure(figsize=(7, 4))
    plt.hist(df["word_count"], bins=50)
    plt.title("Word Count Distribution")
    plt.xlabel("Number of Words"); plt.ylabel("Frequency")
    plt.tight_layout()

```



```

plt.savefig(os.path.join(FIGURE_DIR, "
    word_count_distribution.png"))
plt.close()
return df

def train_model(df):
    df = df[[TEXT_COL, LABEL_COL]].dropna()
    X = df[TEXT_COL].astype(str)
    y = df[LABEL_COL].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)

    model = Pipeline([
        ("tfidf", TfidfVectorizer(stop_words="english",
            lowercase=True, max_features=10000, ngram_range=(1,
                2))),
        ("classifier", MultinomialNB(alpha=0.1))
    ])
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    print("\nModel Used: Multinomial Naive Bayes")
    print("Accuracy:", accuracy)
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred,
        target_names=["Not Spam", "Spam"]))
    print("\nConfusion Matrix:"); print(cm)

    plt.figure(figsize=(5, 4))
    plt.imshow(cm)
    plt.title("Confusion Matrix")
    plt.xlabel("Predicted Label"); plt.ylabel("Actual Label")
    plt.xticks([0,1],["Not Spam","Spam"])
    plt.yticks([0,1],["Not Spam","Spam"])
    for i in range(2):
        for j in range(2):
            plt.text(j, i, cm[i,j], ha="center", va="center")
    plt.tight_layout()

```

```

plt.savefig(os.path.join(FIGURE_DIR, "confusion_matrix.png"
    ))
plt.close()

joblib.dump(model, os.path.join(BASE_DIR, "
    spam_email_detector.pkl"))
print("\nModel saved as spam_email_detector.pkl")
return accuracy

def main():
    create_directory()
    df = pd.read_csv(DATASET_PATH)
    df = perform_eda(df)
    accuracy = train_model(df)
    print(f"\nFinal Accuracy: {accuracy * 100:.2f}%")

if __name__ == "__main__":
    main()

```

Listing 4.1: Training Code for Naive Bayes Spam Email Detection

4.4 Naive Bayes Prediction Code

```

import os
import joblib

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "spam_email_detector.pkl")

def predict_email(email_text, model):
    prediction = model.predict([email_text])[0]
    probability = model.predict_proba([email_text])[0]
    spam_probability = probability[1] * 100
    not_spam_probability = probability[0] * 100
    result = "Spam Email" if prediction == 1 else "Not Spam /
        Ham Email"
    return result, spam_probability, not_spam_probability

def main():
    model = joblib.load(MODEL_PATH)
    print("Spam Email Detector      Naive Bayes")

```

```

print("Type 'exit' to stop.\n")
while True:
    email_text = input("Enter email text: ")
    if email_text.lower() == "exit":
        print("Program closed."); break
    result, spam_prob, ham_prob = predict_email(email_text,
        model)
    print("\nPrediction          :", result)
    print(f"Spam probability    : {spam_prob:.2f}%")
    print(f"Not spam probability: {ham_prob:.2f}%\n")

if __name__ == "__main__":
    main()

```

Listing 4.2: Prediction Code for Naive Bayes Model

4.5 Random Forest Training Code

```

import os
import pandas as pd
import joblib
import matplotlib.pyplot as plt
import numpy as np

from sklearn.model_selection import train_test_split,
    learning_curve
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score,
    classification_report,
    confusion_matrix, roc_curve, auc)

BASE_DIR      = os.path.dirname(os.path.abspath(__file__))
DATASET_PATH  = os.path.join(BASE_DIR, "..", "emails.csv")
FIGURE_DIR   = os.path.join(BASE_DIR, "figures")
TEXT_COL      = "text"
LABEL_COL     = "spam"

def create_directory():
    if not os.path.exists(FIGURE_DIR):

```

```

        os.makedirs(FIGURE_DIR)

def plot_confusion_matrix(cm):
    fig, ax = plt.subplots(figsize=(5, 4))
    im = ax.imshow(cm, cmap="Blues")
    plt.colorbar(im, ax=ax)
    ax.set_title("Confusion Matrix - Random Forest",
                  fontsize=13, fontweight="bold")
    ax.set_xlabel("Predicted Label"); ax.set_ylabel("Actual
    Label")
    ax.set_xticks([0,1]); ax.set_xticklabels(["Not Spam","Spam"
    ])
    ax.set_yticks([0,1]); ax.set_yticklabels(["Not Spam","Spam"
    ])
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i,j], ha="center", va="center",
                    fontsize=14,
                    color="white" if cm[i,j] > cm.max()/2 else
                    "black")
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURE_DIR, "confusion_matrix.png"
    ), dpi=150)
    plt.close()

def plot_feature_importance(pipeline):
    tfidf      = pipeline.named_steps["tfidf"]
    rf_model    = pipeline.named_steps["classifier"]
    feat_names  = tfidf.get_feature_names_out()
    importances = rf_model.feature_importances_
    top_n       = 20
    indices     = np.argsort(importances)[::-1][:top_n]
    top_feat    = [feat_names[i] for i in indices]
    top_scores  = importances[indices]

    fig, ax = plt.subplots(figsize=(9, 6))
    colors    = ["#e74c3c" if s > top_scores.mean()
                  else "#3498db" for s in top_scores]
    ax.barh(range(top_n), top_scores[::-1], color=colors[::-1])
    ax.set_yticks(range(top_n))
    ax.set_yticklabels(top_feat[::-1], fontsize=9)

```

```

ax.set_xlabel("Feature Importance Score")
ax.set_title("Top 20 Important Words - Random Forest",
             fontsize=13, fontweight="bold")
ax.axvline(top_scores.mean(), color="gray", linestyle="--",
           linewidth=1, label="Mean")
ax.legend(); plt.tight_layout()
plt.savefig(os.path.join(FIGURE_DIR, "feature_importance.
    png"), dpi=150)
plt.close()

def plot_roc_curve(pipeline, X_test, y_test):
    y_proba = pipeline.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = auc(fpr, tpr)
    fig, ax = plt.subplots(figsize=(6, 5))
    ax.plot(fpr, tpr, color="#e74c3c", lw=2,
            label=f"Random Forest (AUC = {roc_auc:.4f})")
    ax.plot([0,1],[0,1], color="gray", linestyle="--",
            lw=1, label="Random Guess")
    ax.set_xlabel("False Positive Rate")
    ax.set_ylabel("True Positive Rate")
    ax.set_title("ROC Curve - Random Forest",
                fontsize=13, fontweight="bold")
    ax.legend(loc="lower right"); ax.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURE_DIR, "roc_curve.png"), dpi
        =150)
    plt.close()

def plot_learning_curve(X_train, y_train):
    lc_pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(stop_words="english",
            lowercase=True, max_features=5000, ngram_range
            =(1,1))),
        ("classifier", RandomForestClassifier(
            n_estimators=50, random_state=42, n_jobs=-1))
    ])
    train_sizes, train_scores, val_scores = learning_curve(
        lc_pipeline, X_train, y_train, cv=3,
        train_sizes=np.linspace(0.1, 1.0, 8),
        scoring="accuracy", n_jobs=-1)

```

```

train_mean = train_scores.mean(axis=1)
train_std  = train_scores.std(axis=1)
val_mean   = val_scores.mean(axis=1)
val_std    = val_scores.std(axis=1)

fig, ax = plt.subplots(figsize=(7, 5))
ax.plot(train_sizes, train_mean, "o-",
        color="#e74c3c", label="Training Accuracy")
ax.fill_between(train_sizes,
                train_mean - train_std, train_mean + train_std,
                alpha=0.15, color="#e74c3c")
ax.plot(train_sizes, val_mean, "o-",
        color="#2ecc71", label="Validation Accuracy")
ax.fill_between(train_sizes,
                val_mean - val_std, val_mean + val_std,
                alpha=0.15, color="#2ecc71")
ax.set_xlabel("Training Set Size"); ax.set_ylabel("Accuracy")
ax.set_title("Learning Curve - Random Forest",
            fontsize=13, fontweight="bold")
ax.legend(); ax.grid(alpha=0.3); plt.tight_layout()
plt.savefig(os.path.join(FIGURE_DIR, "learning_curve.png"),
            dpi=150)
plt.close()

def train_model(df):
    df = df[[TEXT_COL, LABEL_COL]].dropna()
    X = df[TEXT_COL].astype(str)
    y = df[LABEL_COL].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(stop_words="english",
                                lowercase=True, max_features=10000, ngram_range
                                =(1,2))),
        ("classifier", RandomForestClassifier(
            n_estimators=100, random_state=42, n_jobs=-1))
    ])
    pipeline.fit(X_train, y_train)

```

```

y_pred    = pipeline.predict(X_test)
accuracy  = accuracy_score(y_test, y_pred)
cm        = confusion_matrix(y_test, y_pred)

print("\nModel Used: Random Forest Classifier")
print("Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
                           target_names=["Not Spam", "Spam"]))
print("\nConfusion Matrix:"); print(cm)

plot_confusion_matrix(cm)
plot_feature_importance(pipeline)
plot_roc_curve(pipeline, X_test, y_test)
plot_learning_curve(X_train, y_train)

joblib.dump(pipeline, os.path.join(BASE_DIR, "spam_rf_model
    .pkl"))
print("\nModel saved as spam_rf_model.pkl")
return accuracy

def main():
    create_directory()
    df        = pd.read_csv(DATASET_PATH)
    accuracy  = train_model(df)
    print(f"\nFinal Accuracy: {accuracy * 100:.2f}%")

if __name__ == "__main__":
    main()

```

Listing 4.3: Training Code for Random Forest Spam Email Detection

4.6 Random Forest Prediction Code

```

import os
import joblib

BASE_DIR    = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH  = os.path.join(BASE_DIR, "spam_rf_model.pkl")

def predict_email(email_text, model):

```

```

prediction = model.predict([email_text])[0]
probability = model.predict_proba([email_text])[0]
spam_probability = probability[1] * 100
not_spam_probability = probability[0] * 100
result = "Spam Email" if prediction == 1 else "Not Spam /
        Ham Email"
return result, spam_probability, not_spam_probability

def main():
    model = joblib.load(MODEL_PATH)
    print("Spam Email Detector - Random Forest")
    print("Type 'exit' to stop.\n")
    while True:
        email_text = input("Enter email text: ")
        if email_text.lower() == "exit":
            print("Program closed."); break
        result, spam_prob, ham_prob = predict_email(email_text,
            model)
        print("\nPrediction      :", result)
        print(f"Spam probability   : {spam_prob:.2f}%")
        print(f"Not spam probability: {ham_prob:.2f}%\n")

if __name__ == "__main__":
    main()

```

Listing 4.4: Prediction Code for Random Forest Model

Chapter 5

Output and Results

5.1 Naive Bayes Training Output

The Naive Bayes model was trained successfully using the provided dataset.

Table 5.1: Naive Bayes Training Output Summary

Output Item	Value
Dataset Shape	5728 rows, 2 columns
Not Spam Emails	4360
Spam Emails	1368
Training Data Size	4582
Testing Data Size	1146
Classifier	Multinomial Naive Bayes
Accuracy	99.04%
Saved Model	spam_email_detector.pkl

5.2 Naive Bayes Classification Report

Table 5.2: Naive Bayes Classification Report

Class	Precision	Recall	F1-score	Support
Not Spam	0.99	0.99	0.99	872
Spam	0.98	0.98	0.98	274
Accuracy	0.99			1146
Macro Average	0.99	0.99	0.99	1146
Weighted Average	0.99	0.99	0.99	1146

5.3 Naive Bayes Confusion Matrix

Table 5.3: Naive Bayes Confusion Matrix

Actual / Predicted	Not Spam	Spam
Not Spam	866	6
Spam	5	269

The confusion matrix shows that the Naive Bayes model correctly classified 866 non-spam emails and 269 spam emails. Only 11 emails were misclassified in total out of 1146 testing samples.

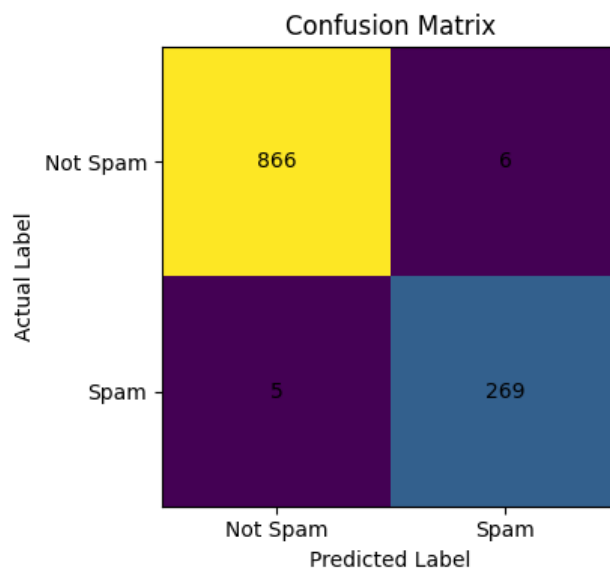


Figure 5.1: Confusion Matrix of Naive Bayes Classifier

5.4 Random Forest Training Output

The Random Forest model was also trained successfully on the same dataset.

Table 5.4: Random Forest Training Output Summary

Output Item	Value
Dataset Shape	5728 rows, 2 columns
Not Spam Emails	4360
Spam Emails	1368
Training Data Size	4582
Testing Data Size	1146
Classifier	Random Forest Classifier
Number of Trees	100
Accuracy	98.34%
AUC Score	0.9983
Saved Model	spam_rf_model.pkl

5.5 Random Forest Classification Report

Table 5.5: Random Forest Classification Report

Class	Precision	Recall	F1-score	Support
Not Spam	0.99	0.99	0.99	872
Spam	0.97	0.96	0.97	274
Accuracy	0.98			1146
Macro Average	0.98	0.98	0.98	1146
Weighted Average	0.98	0.98	0.98	1146

5.6 Random Forest Confusion Matrix

Table 5.6: Random Forest Confusion Matrix

Actual / Predicted	Not Spam	Spam
Not Spam	864	8
Spam	11	263

The Random Forest model correctly classified 864 non-spam emails and 263 spam emails. A total of 19 emails were misclassified out of 1146 testing samples.

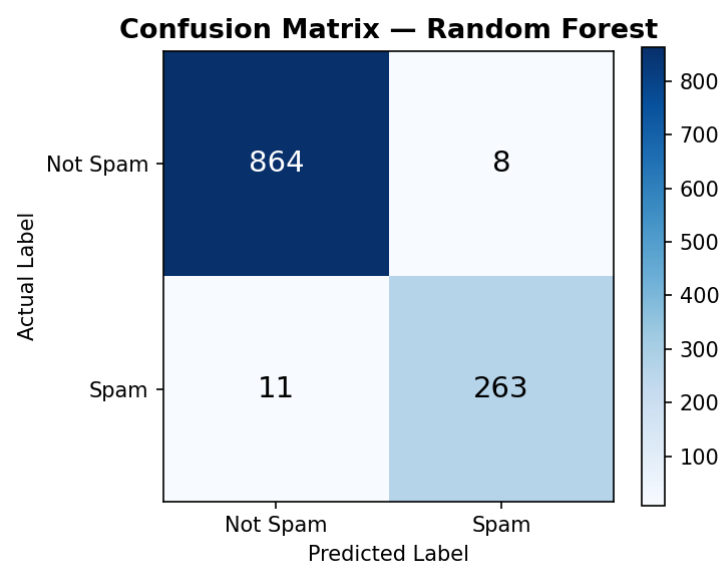


Figure 5.2: Confusion Matrix of Random Forest Classifier

5.7 Sample Prediction Output

Both models were tested with the same new email messages to compare their predictions directly.

Table 5.7: Sample Prediction Results — Both Models

Input Email Text	NB Prediction	RF Prediction
Congratulations! You have won a free iPhone. Click the link now to claim your prize.	Spam	Spam
Sir, I have completed the assignment and submitted it before the deadline.	Not Spam	Not Spam
Urgent! Your account has been selected for a \$1000 cash reward. Click here.	Spam	Spam
Dear student, your class test will be held tomorrow at 10 AM.	Not Spam	Not Spam
Your meeting with the supervisor has been scheduled for Sunday at 2 PM.	Not Spam	Not Spam

Both models produced identical predictions for all sample emails, which confirms that both classifiers learned the correct patterns from the dataset.

Chapter 6

Comparative Analysis

6.1 Performance Comparison

This chapter presents a side-by-side comparison of the Naive Bayes and Random Forest classifiers based on the results obtained from the same dataset and the same train-test split.

Table 6.1: Performance Comparison: Naive Bayes vs Random Forest

Metric	Naive Bayes	Random Forest
Accuracy	99.04%	98.34%
Not Spam Precision	0.99	0.99
Not Spam Recall	0.99	0.99
Not Spam F1-score	0.99	0.99
Spam Precision	0.98	0.97
Spam Recall	0.98	0.96
Spam F1-score	0.98	0.97
Total Misclassified	11	19
AUC Score	—	0.9983
Training Time	Fast	Slower
Feature Interpretability	Low	High

6.2 Analysis and Discussion

From the comparison table, several observations can be made about the two classifiers.

Accuracy: Naive Bayes achieved a higher accuracy of 99.04% compared to Random Forest’s 98.34%. The difference is small but Naive Bayes performs slightly better on this text classification task.

Misclassification: Naive Bayes misclassified only 11 emails out of 1146 test samples, while Random Forest misclassified 19. This further confirms that Naive Bayes is more precise for this dataset.

AUC Score: Random Forest provides an AUC score of 0.9983, which indicates near-perfect discrimination between spam and non-spam. Naive Bayes does not natively provide a meaningful AUC value in this context.

Feature Importance: Random Forest provides feature importance scores that reveal which words most influence the spam classification. This makes the Random Forest model more interpretable and explainable compared to Naive Bayes.

Training Speed: Naive Bayes is significantly faster to train because it only computes word probabilities. Random Forest builds 100 decision trees, which requires more computation time and memory.

Robustness: Random Forest is generally more robust to irrelevant or noisy features because each tree only uses a random subset of features. Naive Bayes assumes feature independence, which may not always hold in real email data.

6.3 When to Use Which Model

Table 6.2: Model Selection Guide

Situation	Naive Bayes	Random Forest
Large dataset, fast training needed	✓	
Need highest accuracy on this data	✓	
Need feature interpretability		✓
Need AUC / probability calibration		✓
Noisy or complex feature space		✓
Real-time email filtering	✓	

Chapter 7

Conclusion

7.1 Concluding Remarks

This assignment successfully developed two spam email detection systems using Naive Bayes and Random Forest classifiers. The provided dataset was analyzed using EDA, and the text data was converted into numerical features using TF-IDF vectorization. The dataset was divided into 80% training data and 20% testing data for both models.

The Naive Bayes model achieved 99.04% accuracy while the Random Forest model achieved 98.34% accuracy. Both models correctly classified almost all test emails and produced identical predictions for all sample inputs. However, Naive Bayes performed slightly better overall on this dataset.

Random Forest provided additional insights through feature importance analysis and an AUC score of 0.9983, which are not available with Naive Bayes. These features make Random Forest more suitable for scenarios where model explainability is required.

7.2 Limitations

Although both models performed well, there are some limitations:

- Both models were trained on a fixed dataset, so performance may vary on completely different datasets.
- The systems only classify text-based emails and do not analyze attachments or URLs.
- Both implementations run in a command-line interface.
- Neither model may detect advanced phishing emails that use very natural language.
- Random Forest requires significantly more computation time and memory than Naive Bayes.

7.3 Future Scopes

Both projects can be improved in the future in the following ways:

- A web-based interface can be developed for easier user interaction.
- More datasets can be added to improve generalization.
- Deep learning models such as LSTM or BERT-based transformer models can be tested and compared with these classifiers.
- Email attachments and URLs can be analyzed for better spam and phishing detection.
- A combined ensemble model using both Naive Bayes and Random Forest can be explored for even higher accuracy.